MobileSec: A Microservices-Based Intelligent Platform for Automated Android APK Vulnerability Analysis and Remediation.

Abdelali Ayoujil[a], Mohamed Lachgar[a,b], Omar Achbarou[c], Mohamedou Cheikh Tourad[d]

[a] *Cadi Ayyad University, I2IS Laboratory, Marrakech, Morocco*
[b] *Cadi Ayyad University, Higher Normal School, Computer Science Department, Marrakech, Morocco*
[c] *Cadi Ayyad University, National School of Applied Sciences, IMSC Laboratory, Marrakech, Morocco*
[d] *University of Nouakchott, Faculty of Sciences and Techniques, Al-Khawarizmi, Nouakchott, Mauritania*

Abstract

MobileSec is an open-source platform for the static security analysis of Android application packages (APKs), designed around a modular microservices architecture. The platform automates security assessment by combining APK preprocessing with specialized analysis services that detect issues such as cryptographic misuse, hard-coded secrets, and insecure network configurations. To help users focus on the most important findings, MobileSec incorporates a LightGBM-based model for vulnerability prioritization, producing ranked results with confidence estimates. The platform also includes an AI-assisted remediation component that provides vulnerability explanations, mitigation guidance, and Java/Kotlin-oriented fix suggestions through large language model integration. Its containerized design supports scalable deployment and reproducible experimentation, making the platform useful for researchers, security analysts, and developers working on Android application security.

Keywords: Android security; APK static analysis; machine learning; AI-assisted remediation

Metadata

Table 1: Code metadata

Nr	Code metadata description	Metadata
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	`https://github.com/ayoujil200/mobilesec`
C3	Permanent link to reproducible capsule	`https://github.com/ayoujil200/mobilesec`
C4	Legal code license	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Python, Kotlin, JavaScript, Docker
C7	Compilation requirements, operating environments and dependencies	Docker Engine, Python 3.10+, Node.js 18+, Androguard, LightGBM, Flask, Spring Boot, React
C8	If available, link to developer documentation/manual	N/A
C9	Support email for questions	`a.ayoujil.ced@uca.ac.ma`

1. Motivation and significance

Android applications make up a major portion of the mobile software ecosystem and, as a result, are common targets for security threats such as malware injection, credential leakage, improper use of cryptographic mechanisms, and insecure network communication [1, 2]. Static analysis has long served as a scalable method for identifying these vulnerabilities, but many existing Android security analysis tools remain limited in scope. They often concentrate on specific vulnerability types, depend heavily on rule-based detection methods, or generate large numbers of findings without clear prioritization [3, 4]. In addition, many tools do not provide built-in support for interpreting vulnerabilities or guiding remediation, which forces analysts to manually connect results across different and often inconsistent toolchains. Together, these shortcomings reduce scalability, reproducibility, and efficiency in both research-oriented and industry-based Android security assessments.

The development of *MobileSec* is motivated by the need for an integrated and extensible software platform that consolidates static APK analysis, vulnerability prioritization, and remediation support within a unified framework. The software is designed to address the scientific problem of transforming low-level static analysis outputs into actionable, ranked security insights. By combining multiple analysis techniques into a modular pipeline, MobileSec reduces fragmentation in Android security workflows and enables systematic

experimentation with vulnerability detection and ranking strategies, building on foundations provided by widely used tools such as Androguard [5] and benchmark suites like DroidBench [6].
MobileSec contributes to the process of scientific discovery in mobile security research by supporting reproducible and extensible experimentation. Its microservices-based architecture allows individual analysis components—including cryptographic misuse detection, secret leakage analysis, and network security inspection—to be independently developed, evaluated, and replaced. This design facilitates comparative studies of static analysis techniques and enables the integration of machine learning–based methods for vulnerability assessment [7]. In particular, the inclusion of a LightGBM-based classifier [8] enables data-driven prioritization of detected vulnerabilities, allowing researchers to study the impact of different feature representations and ranking models on vulnerability triage effectiveness.
From both an experimental and user-oriented perspective, MobileSec serves as a fully integrated end-to-end static analysis pipeline. Users submit Android APK files through a web-based interface, after which the platform automatically performs static analysis using specialized services. The resulting findings are then aggregated, scored, and presented through interactive dashboards and structured reports that can be exported for deeper examination. In addition, an optional AI-assisted remediation module provides explanatory insights and code-level fix suggestions, allowing researchers to evaluate automated vulnerability explanation and mitigation support enabled by large language models. [9, 10]
MobileSec expands on and works alongside existing Android security analysis tools and frameworks, including decompilation-based analyzers, rule-based static scanners, and approaches that follow industry standards such as the OWASP Mobile Application Security Verification Standard (MASVS) [11]. While many current solutions concentrate mainly on identifying vulnerabilities, MobileSec gives greater attention to modular deployment, automated vulnerability prioritization, and built-in remediation support. Together, these features make the software valuable both as a practical research platform for Android security analysis and as a tool that can be applied in real-world security assessment workflows.

2. Software description

MobileSec is a containerized platform built on a microservices architecture for assessing the security of Android applications. It is designed to analyze uploaded APK and AAB files, store both intermediate and final analysis results in MongoDB, and provide outputs that are suitable for both machine

processing and human interpretation in research and operational settings. The current implementation integrates deterministic static analysis checks, service-level orchestration, model-driven vulnerability prioritization, and AI-assisted remediation guidance.

In practice, MobileSec structures its analysis workflow as a staged pipeline that includes artifact ingestion, APK preprocessing, specialized security analysis, vulnerability aggregation, prioritization, remediation recommendation, and report export. This design allows each security module, such as crypto, secrets, network, ML, and reporting, to evolve independently without the need for a monolithic redesign.

2.1. Software architecture

The figure 1 gives an overview of the MobileSec architecture, highlighting how its main services work together and how data flows through the analysis process.

MobileSec adopts a modular microservices architecture in which each major stage of the analysis pipeline is implemented as a separate independent service. This design supports separation of concerns, scalability, fault isolation, and parallel processing of security analyses. Communication between services is handled through a mix of synchronous REST-based requests and asynchronous event-driven messaging.

User interaction takes place through a web-based Frontend Dashboard that allows users to submit APKs, monitor scan progress in real time, and view structured analysis results. The dashboard communicates only with the API Gateway, which preserves the abstraction of the underlying internal architecture.

The architecture is organized around the following main services:

- **API Gateway**: At the system's entry point, an API Gateway serves as the central interface between the frontend dashboard and the backend services. It is responsible for validating incoming requests, handling APK uploads, routing traffic to the appropriate analysis components, and exposing structured API documentation. By separating client interactions from the internal services, the gateway improves maintainability and helps enforce controlled access to system resources.

- **APK Scanner**: The main static analysis process begins in the APK Scanner service. This component automatically disassembles submitted APK files and extracts relevant artifacts, such as bytecode representations, manifest declarations, permissions, exported components, and embedded resources. It also detects network endpoints and configuration settings. After preprocessing is completed, the scanner emits

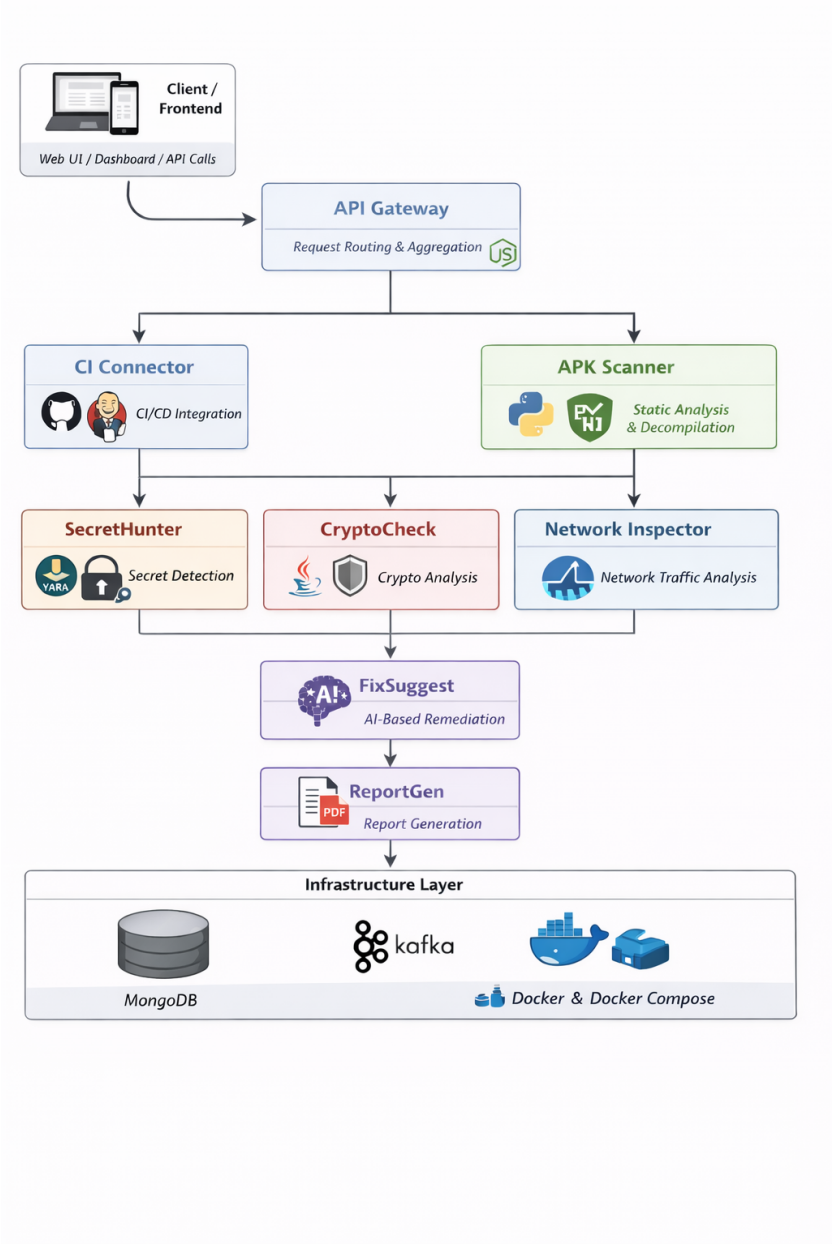

Figure 1: Overview of the MobileSec architecture illustrating service interaction and data flow.

structured events to the message broker, which then triggers the specialized analysis services.

- **SecretHunter**: The SecretHunter service is responsible for detecting hardcoded credentials and other sensitive information. It examines decompiled source artifacts using pattern-based and rule-driven techniques to uncover API keys, authentication tokens, private keys, and other embedded secrets. The service produces structured findings that are supplemented with relevant contextual metadata.

- **CryptoCheck**: Cryptographic implementations are analyzed by the CryptoCheck service. This component reviews code structures to identify insecure algorithm choices, weak hash functions, poor key management practices, hardcoded salts, and unsafe initialization vectors. By concentrating specifically on cryptographic correctness, it delivers focused analysis in one of the most security-sensitive areas of mobile application security.

- **Network Inspector**: Security issues related to networking are evaluated by the Network Inspector. This service reviews extracted endpoints and communication settings to identify insecure transport protocols, missing certificate validation, improper certificate pinning, and possible communication with untrusted domains.

- **Machine Learning Model**: In addition to rule-based analysis, MobileSec includes a dedicated Machine Learning Model service. This component classifies applications and vulnerability patterns using trained models to detect suspicious behavior or malware-related characteristics that may be missed by deterministic static rules. By relying on patterns learned from labeled datasets, the model helps broaden overall detection coverage.

- **FixSuggest**: To support remediation, the FixSuggest service produces structured explanations and recommended fixes for identified vulnerabilities. It combines predefined remediation templates with large language model (LLM)-based generation to create contextualized, code-level guidance, helping reduce the effort required to understand and address security issues.

- **MongoDB**: All findings generated by the analysis services are aggregated and stored in a centralized MongoDB database. This persistent storage supports traceability of scan results, reproducibility of experiments, and historical comparison across different application versions.

- **Report Generation (ReportGen)**: A dedicated Report Generation (ReportGen) service compiles vulnerability findings into structured outputs, including machine-readable summaries and stakeholder-focused documents such as PDF and HTML reports. In this way, the service converts raw analysis results into more understandable and actionable security assessments.

- **CI Connector**: To support automated DevSecOps practices, a CI Connector service allows the platform to integrate with continuous integration and deployment pipelines. As a result, security scans can be triggered automatically during build processes, embedding security assessment directly into the software development lifecycle.

- **Apache Kafka and Docker**: Communication between services is supported by Apache Kafka, which serves as the platform's messaging backbone. Kafka enables asynchronous, decoupled processing so that multiple analysis services can consume scan events independently and run in parallel. The entire platform is containerized with Docker, which helps ensure reproducible deployments and consistent runtime environments across both development and production.

The overall process, beginning with APK submission and continuing through distributed analysis, intelligent classification, remediation support, and report generation, is illustrated in Fig. 1. MobileSec's modular design also supports extensibility, making it possible to add new security services without affecting the existing components.

2.2. Machine Learning-Based Vulnerability Prioritization

MobileSec includes a machine learning module for prioritizing detected vulnerabilities and predicting the most appropriate remediation category. The model is implemented as a LightGBM gradient-boosted decision tree classifier. Its input consists of aggregated vulnerability features extracted from the outputs of CryptoCheck, SecretHunter, and NetworkInspector. The output is a ranked list of remediation classes with confidence scores.

The learning task is formulated as a multiclass classification problem. For each scan, the target label is the primary fix category, denoted as `fix_category`. When FixSuggest recommendations are available, the first generated recommendation is used as the primary label. Otherwise, a deterministic rule-based labeling strategy is used to assign a remediation class according to the dominant vulnerability type.

Table 2: Features used by the LightGBM prioritization model.

Feature group	Features
Cryptography	`crypto_total_vulns`, `crypto_high`, `crypto_medium`, `crypto_low`, `crypto_info`, `crypto_weak_cipher`, `crypto_weak_hash`, `crypto_insecure_random`, `crypto_weak_rsa`, `crypto_unique_cwes`
Secrets	`secrets_count`, `secrets_api_keys`, `secrets_passwords`, `secrets_tokens`, `secrets_aws_keys`, `secrets_other`, `secrets_unique_types`
Network	`network_findings`, `network_endpoints`, `network_http_issues`, `network_cert_issues`, `network_domain_issues`
Aggregated	`total_vulnerabilities`, `severity_score`

The model predicts one of thirteen remediation categories, including weak cipher replacement, weak hash replacement, insecure random number generation, exposed API keys, hardcoded passwords, insecure HTTP usage, certificate validation issues, and cases where no critical issue is detected.

2.3. Software functionalities

MobileSec currently provides the following implemented capabilities:

1. **APK ingestion and scan lifecycle management:** Handling APK uploads and managing the scan lifecycle, including scan ID generation, stage-level status tracking, and monitoring the health of individual services.
2. **APK structural and manifest analysis:** Extraction of package metadata, permissions, exported components, debuggable and cleartext settings, certificate details, DEX statistics, and potential endpoint candidates.
3. **Multi-domain vulnerability detection:** Specialized security findings derived from cryptographic analysis, secret leakage detection, and network security inspection.
4. **Asynchronous orchestration:** Coordinated fan-out execution across services through REST-based notifications and Kafka topic consumption, with scan-state data persisted in MongoDB to support reproducibility.

5. **Data-driven prioritization:** LightGBM-based prediction of remediation categories, along with confidence-ranked recommendations generated from engineered features collected across multiple services.
6. **AI-assisted remediation:** MASVS-guided and LLM-enhanced vulnerability explanations, prioritized remediation guidance, and patch suggestions tailored for Java and Kotlin.
7. **Report generation and export:** Findings are consolidated, cleaned, and scored before being presented in the dashboard and exported in PDF, JSON, and SARIF formats.
8. **DevSecOps integration support:** generation of CI templates and trigger endpoints to embed security scanning into automated build pipelines.

2.4. Sample code snippets analysis

The function `perform_complete_scan` illustrates a complete static analysis pipeline for an Android APK. It orchestrates multiple stages, including structural bytecode analysis, manifest parsing using `ManifestParser`, permission-based risk assessment with `PermissionsAnalyzer`, and extraction of sensitive data such as API keys and network endpoints. The results are consolidated into a comprehensive security score and stored for further processing or notification to downstream services. This snippet highlights the integration of various tools and analyses into a single automated workflow for Android security evaluation.

```
def perform_complete_scan(scan_id, apk_path):
    """
    Orchestrates the multi-stage static analysis pipeline for
    ↪ an Android APK.
    Combines manifest parsing, bytecode analysis, and risk
    ↪ scoring.
    """
    results = {
        'scan_id': scan_id,
        'timestamp': datetime.utcnow().isoformat(),
        'status': 'in_progress'
    }

    try:
        # 1. Structural Analysis (Bytecode/Resources)
        # Uses Androguard to extract basic metadata and
        ↪ compiled strings
        androguard = AndroguardWrapper(apk_path)
        androguard.load_apk()
        basic_info = androguard.get_basic_info()

```

```
        # 2. Manifest & Decompilation (External Tool Integration)
        # Invokes Apktool to reconstruct original XML configurations
        manifest_parser = ManifestParser(apk_path)
        decompiled_path = os.path.join(DECOMPILED_FOLDER, scan_id)
        manifest_parser.decompile_apk(decompiled_path)
        manifest_data = manifest_parser.parse_manifest()

        # 3. Behavioral Risk Assessment (Permissions)
        # Evaluates the permission model and calculates a weighted risk score
        analyzer = PermissionsAnalyzer()
        perm_analysis = analyzer.analyze_permissions(basic_info.get('permissions', []))

        # 4. Static Data Extraction (Secrets & Endpoints)
        # Performs regex-based pattern matching on Dalvik strings
        results.update({
            'package_name': basic_info['package_name'],
            'debuggable': manifest_data.get('debuggable', False),
            'permissions_risk': perm_analysis['risk_score'],
            'endpoints': androguard.extract_network_endpoints(),
            'secrets': androguard.get_api_keys(),
            'certificate': androguard.get_certificate_info()
        })

        # 5. Global Security Scoring
        # Synthesizes all findings into a normalized security metric (0-100)
        results['security_score'] = calculate_security_score(results)
        results['status'] = 'completed'

        # Persistent Storage & Inter-service Communication
        mongodb_client.save_scan_results(scan_id, results)
        produce_scan_event(scan_id, apk_path) # Notify downstream microservices (Kafka)

        return results

    except Exception as e:
        logger.error(f"Orchestration failure: {e}")
        return {'scan_id': scan_id, 'status': 'failed', 'error': str(e)}
```

3. Illustrative examples

This section presents representative usage scenarios demonstrating how MobileSec is used in practice to perform Android application security analysis, vulnerability prioritization, and remediation support.

3.1. End-to-end static analysis of an Android APK

In a typical workflow, a developer or security analyst uploads an Android application package (APK) through the web-based interface. The frontend includes a drag-and-drop upload feature and shows scan progress in real time. After submission, the CI-Connector service manages the analysis pipeline by routing the APK to the appropriate scanning services.

The APK-Scanner service decompiles the application and extracts key resources, bytecode, and configuration files. The extracted artifacts are then examined in parallel by specialized microservices. CryptoCheck identifies cryptographic misuse, such as weak algorithms or insecure modes. SecretHunter searches for hard-coded secrets, including API keys and credentials. NetworkInspector reviews network configurations to uncover insecure communication patterns and SSL/TLS misconfigurations.

The scan results are then consolidated and displayed on the main dashboard, which provides an overview of detected vulnerabilities by category, severity level, and scan status (Fig. 2). Users can explore each category in more detail to review specific findings, including file paths, line references, and vulnerability descriptions. Overall, this workflow allows a full static security assessment of an APK to be carried out within a single platform.

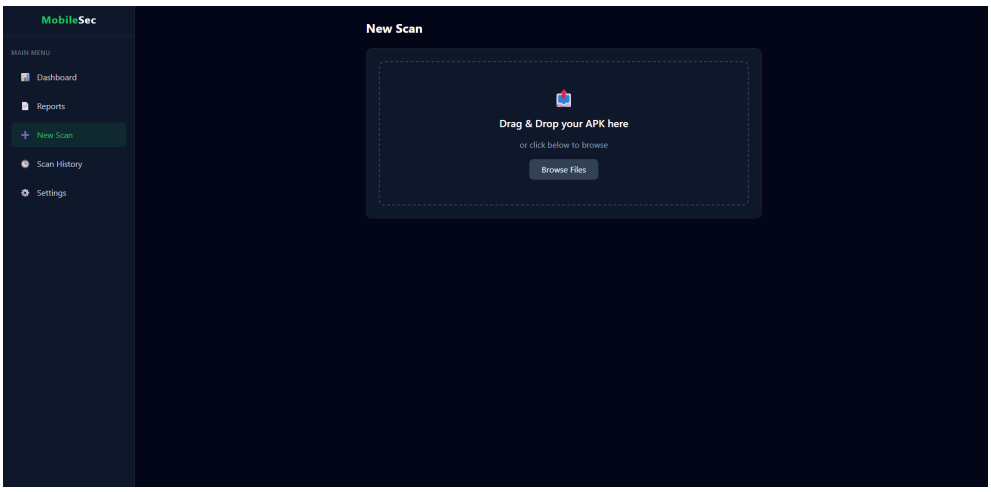

Figure 2: MobileSec dashboard summarizing detected vulnerabilities by category, severity level, and scan status after static APK analysis.

3.2. Vulnerability prioritization and AI-assisted remediation

After the static analysis is complete, MobileSec uses a LightGBM-based machine learning model to prioritize the detected vulnerabilities. The model assigns confidence scores and ranks the findings based on their estimated security impact. The frontend then displays a prioritized list of vulnerabilities, helping users quickly focus on the most critical issues. (Fig. 3).

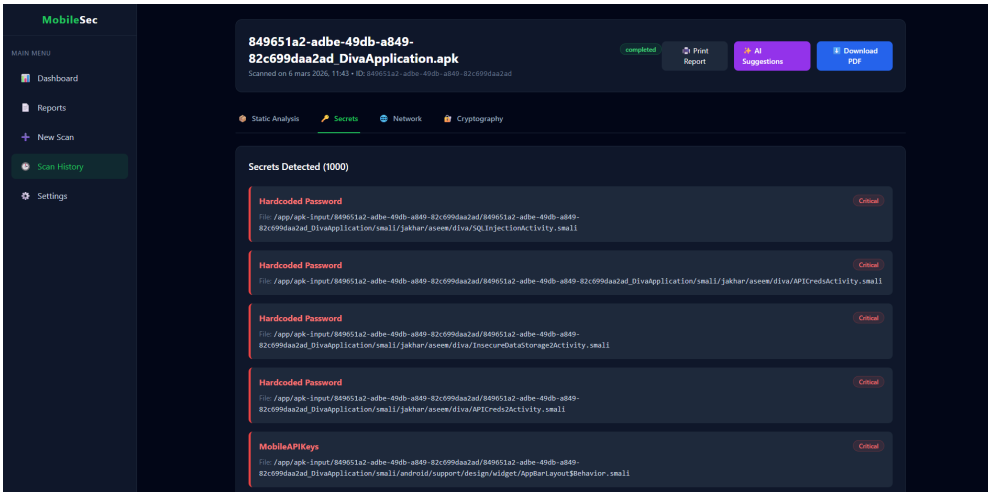

Figure 3: Prioritized vulnerability list ranked using the LightGBM-based machine learning model, highlighting critical findings.

For selected vulnerabilities, users can request AI-assisted remediation through the FixSuggest service. This component uses large language models accessed through the OpenRouter API to produce structured explanations of the security issue, step-by-step mitigation guidance, and complete Java or Kotlin code-level fix recommendations (Fig. 4). The generated patches include the necessary imports and follow platform-specific development practices.

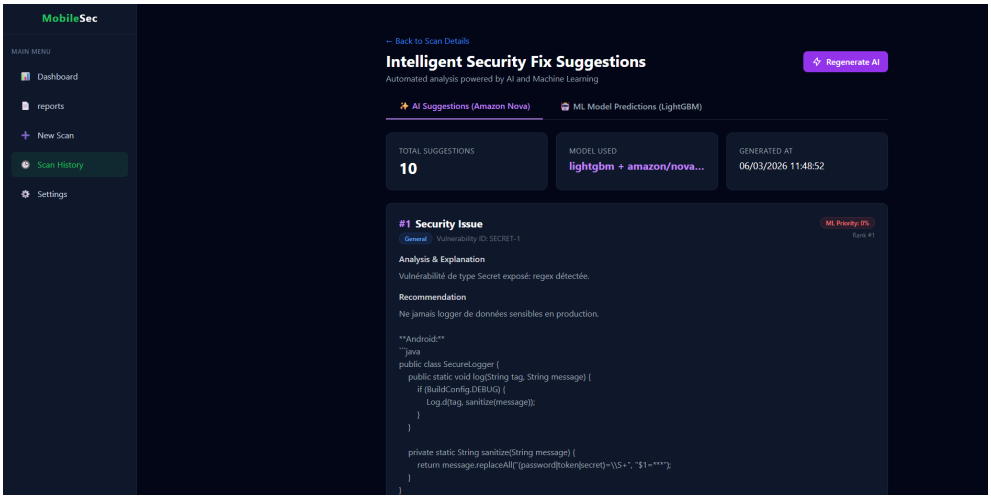

Figure 4: AI-assisted remediation interface showing vulnerability explanation and generated Java/Kotlin code-level fix suggestions.

The platform pairs machine learning–driven prioritization with AI-generated remediation guidance, helping developers concentrate on the most critical vulnerabilities while getting practical recommendations that reduce the need for manual analysis.

3.3. Automated security report generation

MobileSec also makes it possible to automatically generate professional security reports. From the scan detail view, users can launch the ReportGen service to create a PDF report that summarizes all identified findings. The report contains an executive summary, a breakdown of vulnerabilities by category and severity, and detailed descriptions of each individual issue. Generated reports are saved and can be accessed through a dedicated report history interface, where users are able to view, download, or delete earlier reports (Fig. 5). This feature helps teams communicate results to non-technical stakeholders and supports the use of MobileSec within formal security assessment and audit processes.

Explanation of how each penalty is calculated. The generated report also presents an overall security score that summarizes the main risks identified during analysis. The score is calculated using a penalty-based approach in which the computation starts from 100 and subtracts weighted penalties for each detected issue category, including debuggable builds, cleartext traffic settings, risky permissions, insecure endpoints, hardcoded secrets, and exported components. To keep the score easy to read and compare across scans, the final value is clipped to the range $[0, 100]$.

$$\begin{aligned}\text{Security Score} = \max\Big(0,\ 100 &- \text{debuggable\_penalty} \\ &- \text{cleartext\_penalty} \\ &- \text{permissions\_penalty} \\ &- \text{insecure\_endpoints\_penalty} \\ &- \text{hardcoded\_secrets\_penalty} \\ &- \text{exported\_components\_penalty}\Big)\end{aligned} \tag{1}$$

The different penalty terms are defined as follows:

Debuggable penalty. If the application is set to debug mode, a fixed deduction of 25 points is enforced:

$$\text{debuggable\_penalty} = \begin{cases} 25 & \text{if the application is configured as debuggable} \\ 0 & \text{otherwise} \end{cases}$$

Cleartext traffic penalty. If unencrypted traffic is permitted, a fixed penalty of 15 points is imposed:

$$\text{cleartext\_penalty} = \begin{cases} 15 & \text{if cleartext traffic is allowed} \\ 0 & \text{otherwise} \end{cases}$$

Permissions penalty. Let R_{perm} represent the permission risk score calculated by the permissions analysis component. The corresponding deduction is:

$$\text{permissions\_penalty} = \min\left(\left\lfloor \frac{R_{perm}}{5} \right\rfloor, 20\right)$$

Thus, every five permission risk points result in a one-point deduction, capped at a maximum penalty of 20.

Insecure endpoints penalty. Let $N_{endpoints}$ represent the number of insecure endpoints identified during the analysis. The penalty is calculated as:

$$\text{insecure\_endpoints\_penalty} = \min\left(2 \times N_{endpoints}, 15\right)$$

Therefore, each insecure endpoint adds two penalty points, with the total penalty capped at 15.

Hardcoded secrets penalty. Let $N_{secrets}$ represent the number of hardcoded or potentially exposed secrets identified. The corresponding penalty is:

$$\text{hardcoded\_secrets\_penalty} = \min\left(3 \times N_{secrets}, 20\right)$$

In this scenario, each detected secret adds three penalty points, with the total deduction capped at 20.

Exported components penalty. Let $N_{exported}$ represent the number of exported application components. No penalty is imposed when this value is five or less. Once it exceeds that threshold, the deduction is calculated as:

$$\text{exported\_components\_penalty} = \begin{cases} 0 & \text{if } N_{exported} \leq 5 \\ \min\left(N_{exported} - 5, 10\right) & \text{if } N_{exported} > 5 \end{cases}$$

This indicates that each exported component beyond the initial five results in a one-point deduction, capped at a maximum of 10.

Once all penalties have been combined, the final score is bounded below at 0, meaning any negative value is set to 0. The implementation then assigns a qualitative grade based on the numeric score: *A / Excellent* for scores of 80 or higher, *B / Good* for scores between 60 and 79, *C / Fair* for scores between 40 and 59, *D / Poor* for scores between 20 and 39, and *F / Critical* for scores below 20.

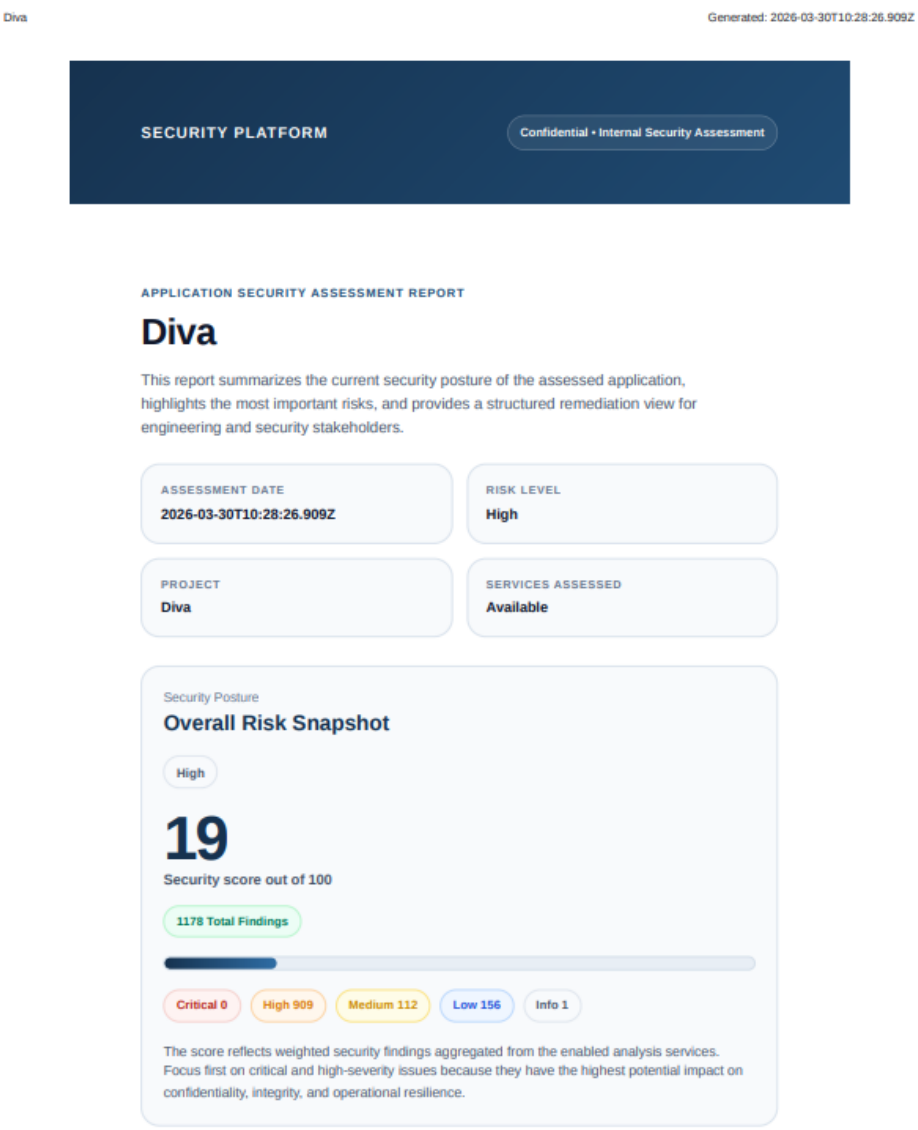

Figure 5: MobileSec security report generation

4. Impact

MobileSec shifts Android security analysis from a fragmented sequence of separate tools toward a more unified and reproducible workflow. Rather than requiring analysts to move manually between decompilers, rule-based scanners, prioritization scripts, and reporting utilities, the platform brings these stages together within a single environment. This reduces practical overhead during experimentation and makes repeated analyses easier to reproduce.

The platform also aligns with standardization efforts such as SARIF for the exchange of static-analysis results and with mobile-security guidance such as OWASP MASVS and MASTG, both of which emphasize structured and repeatable security assessment practices for mobile applications [12, 11, 13].
From a research perspective, MobileSec provides a useful basis for studying how findings produced by different analyzers interact and how prioritization models behave under different configurations. Because outputs from multiple services are collected within the same pipeline, researchers can compare detector combinations, orchestration strategies, and ranking approaches without rebuilding the workflow for each experiment. This is especially relevant for benchmark-driven evaluation using resources such as DroidBench and Ghera [6, 14].
The platform also improves day-to-day experimental practice. Intermediate artifacts, including manifest-derived information, certificate metadata, permissions, and extracted endpoints, are stored for reuse, which reduces repeated preprocessing during iterative studies. In addition, the asynchronous orchestration model allows multiple specialized analyzers to run in parallel, which is consistent with modern DevSecOps-oriented security workflows [15, 16]. The LightGBM-based prioritization layer further moves the workflow beyond raw finding counts toward confidence-aware triage, which makes the resulting analyses more useful for practical decision-making [8].
For developers and security analysts, MobileSec supports an integrated scan-to-report workflow. Users can submit an APK, inspect prioritized findings, review remediation suggestions, and export structured reports without switching between disconnected tools. This reduces the time spent on normalizing outputs, reconciling severity levels, and preparing security documentation. Support for JSON and SARIF also makes the results easier to integrate into downstream platforms and audit processes [12].
Beyond academic use, MobileSec also has clear translational value for industrial settings. Its separation into scanning, detection, prioritization, remediation, and reporting services matches common deployment patterns in DevSecOps and enterprise software assurance, where gradual adoption and CI/CD integration are important practical considerations [15, 16, 17]. In this sense, the platform is not only a research prototype, but also a useful foundation for real-world software security workflows.

4.1. Comparative analysis with MobSF, BeVigil, and Yaazhini

To better situate *MobileSec* within the broader mobile application security landscape, we compare it with three representative solutions: *MobSF*, *BeVigil*, and *Yaazhini* (currently presented as the Vooki Android App Scanner). These tools were chosen because they reflect different usage mod-

els within the mobile security ecosystem: open-source automated analysis (*MobSF*), large-scale application intelligence and risk discovery (*BeVigil*), and Android/APK-focused vulnerability scanning with API-oriented capabilities (*Yaazhini/Vooki*).

MobSF is widely recognized as a comprehensive mobile security testing framework that supports both static and dynamic analysis, cross-platform application assessment, and exportable security reports. Based on its public documentation, it supports Android, iOS, and Windows applications, provides both static and dynamic testing features, and produces reports containing severity classifications, affected files, and remediation guidance. *BeVigil*, in contrast, is positioned more as a mobile application security intelligence and search platform. Its public scanning interface highlights app risk scores, large-scale metadata search, code browsing, secret detection, and on-demand APK scanning. *Yaazhini/Vooki AAS* primarily targets Android APK and API vulnerability analysis, reverse engineering, URL discovery, and detailed vulnerability reporting accompanied by risk ratings and mitigation-focused explanations.

Compared with these platforms, *MobileSec* is designed as a microservices-based research and engineering platform focused on automated Android APK analysis. Its current implementation integrates APK preprocessing, specialized static analyzers for cryptographic misuse, secret exposure, and network-related weaknesses, a LightGBM-based prioritization module, AI-assisted remediation generation, PDF/JSON/SARIF-oriented reporting, and CI-oriented integration support. Rather than concentrating solely on vulnerability identification, *MobileSec* explicitly links vulnerability detection with prioritization, explanation, remediation, and report generation in a unified end-to-end workflow.

Table 3: **Qualitative comparison of MobileSec with representative mobile application security analysis platforms**

Criterion	**MobileSec**	**MobSF[1]**	**BeVigil[2]**	**Yaazhini / Vooki AAS[3]**
Primary scope	Android APK security analysis platform	Mobile application security testing framework	Mobile app security search and risk-intelligence platform	Android APK and related API vulnerability scanner

Continued on next page

Criterion	MobileSec	MobSF[1]	BeVigil[2]	Yaazhini / Vooki AAS[3]
Architecture	Microservices-based	Integrated framework	Cloud/platform-oriented search and reporting workflow	Scanner-oriented product workflow
Platform emphasis	Android	Android, iOS, Windows	Mobile application ecosystem discovery and risk visibility	Android APK and related APIs
Static analysis	Yes	Yes	Partial / report-oriented exposure analysis	Yes
Dynamic analysis	Partial pipeline support through specialized services; mainly static in the current implementation	Yes	Not prominently documented as a dynamic runtime analysis framework	API traffic capture / scanning workflow documented
Cryptography checks	Yes	Broad security checks	Exposed through reported findings when detected	Broad vulnerability scanning reported
Secret detection	Yes	Yes	Yes, through extracted findings and exposed app data	Broad vulnerability scanning reported
Network analysis	Yes	Yes	Risk/report-oriented visibility over endpoints and app metadata	Yes, especially through API scanning

Continued on next page

Criterion	MobileSec	MobSF[1]	BeVigil[2]	Yaazhini / Vooki AAS[3]
ML-based prioritization	Yes	Not highlighted as a core documented feature	Risk-score centric platform view	Not highlighted as a core documented feature
AI-assisted remediation	Yes	Remediation guidance / documented findings	Not highlighted as a core documented feature	Mitigation / explanation support reported in product pages
Report generation	PDF reports and structured outputs	HTML / PDF / JSON export	Security reports and risk score views	Downloadable vulnerability reports
CI / DevSecOps orientation	Yes	Yes	More search/intelligence oriented than CI-centric	Not prominently emphasized in public documentation
Research extensibility	High, due to modular microservices	Moderate to high	Lower for self-hosted experimental modification	Moderate

The comparison shows that *MobileSec* is not intended to simply mirror existing mobile scanning solutions. Rather, it establishes a distinct role by bringing together: (i) modular Android-focused security analysis, (ii) machine-learning-based vulnerability prioritization, (iii) AI-assisted remediation support, and (iv) integrated reporting and orchestration. These characteristics make it especially well suited for reproducible experimentation and for investigating how vulnerability detection, prioritization, and remediation assistance can be consolidated within a single platform.
From a practical standpoint, *MobSF* continues to serve as a strong general-purpose benchmark due to its maturity, cross-platform coverage, and combined static and dynamic analysis capabilities. *BeVigil* is more appropriately

[1] `https://mobsf.github.io/Mobile-Security-Framework-MobSF/`
[2] `https://bevigil.com/`
[3] `https://www.vegabird.com/yaazhini/`

viewed as a large-scale discovery and intelligence platform for exploring mobile application risk, rather than as a self-contained, research-oriented analysis pipeline. *Yaazhini/Vooki AAS* offers a more focused Android scanning environment with APK reverse engineering and API-centered security assessment, making it closer to *MobileSec* in terms of Android specialization. However, it does not place the same emphasis on microservice extensibility, machine-learning-based prioritization, or LLM-assisted remediation. Overall, the comparison indicates that the novelty of *MobileSec* lies less in vulnerability detection alone and more in its integration of intelligent ranking, remediation assistance, modular deployment, and a reproducible end-to-end workflow.

4.2. Quantitative Comparison With Existing Tools

In addition to the qualitative comparison, MobileSec was compared quantitatively with MobSF, BeVigil, and Yaazhini. The comparison considers runtime, number of findings, detection coverage, and reproducibility. For benchmark APKs with available ground truth, precision, recall, and F1-score are reported.

Table 4: Quantitative comparison of MobileSec with existing Android security analysis tools.

Tool	**APKs**	**Avg. runtime**	**Findings/APK**	**Precision**	**Recall**	**F1**
MobileSec	TODO	TODO	TODO	TODO	TODO	TODO
MobSF	TODO	TODO	TODO	TODO	TODO	TODO
BeVigil	TODO	TODO	TODO	TODO	TODO	TODO
Yaazhini	TODO	TODO	TODO	TODO	TODO	TODO

5. Conclusions

This work introduced *MobileSec*, a modular platform for Android security assessment that brings together static APK analysis, specialized vulnerability detection, machine learning–based prioritization, AI-assisted remediation, and multi-format reporting within a single workflow. In doing so, the platform addresses a practical limitation in current mobile security practice: while many tools are able to identify issues, far fewer offer a complete path from raw findings to prioritized and actionable remediation.

From an engineering standpoint, MobileSec shows that a microservices-based architecture can support diverse security analyses while maintaining reproducibility and deployment portability through containerization. The current implementation incorporates domain-specific components for cryptography,

secret exposure, and network-related risk, all coordinated through a structured scan pipeline with persistent result storage and asynchronous messaging. This design makes the platform valuable not only for operational scanning, but also for controlled research settings in which individual services, models, or orchestration approaches can be swapped out and assessed independently.

One of the main contributions of the platform is its combination of deterministic analysis results with data-driven triage and remediation support. The LightGBM-based prioritization service converts findings from multiple sources into ranked recommendations accompanied by confidence estimates, while the FixSuggest component enhances findings with MASVS-aligned and LLM-assisted remediation guidance. Together, these capabilities help reduce analyst burden and make static analysis results more practical and usable.

MobileSec also improves communication and integration. Support for PDF, JSON, and SARIF report generation enables both human-readable and machine-consumable outputs, while CI-oriented endpoints make it easier to integrate the platform into DevSecOps pipelines. These features broaden the software's usefulness across research environments, educational contexts, and industrial security operations.

Future work will concentrate on large-scale empirical benchmarking, stronger evaluation datasets, expanded detector coverage, and more rigorous assessment of remediation effectiveness, including metrics such as fix acceptance rate, time to remediate, and false-positive impact. Other directions include stronger policy enforcement for enterprise deployments, better explainability for ranking decisions, and continuous retraining of models using historical scan feedback. Overall, MobileSec offers a strong and extensible foundation for reproducible Android security analysis and practical vulnerability management.

References

[1] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel, FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps, *ACM SIGPLAN Notices*, vol. 49, no. 6, pp. 259–269, 2014.

[2] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. Sheth, TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones, *ACM Transactions on Computer Systems*, vol. 32, no. 2, 2014.

[3] Y. Feng, S. Anand, I. Dillig, and A. Aiken, Apposcopy: Semantics-based detection of Android malware through static analysis, in *Proceedings of the ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2014.

[4] L. Li, A. Bartel, T. F. Bissyandé, J. Klein, Y. Le Traon, and S. Arzt, IccTA: Detecting inter-component privacy leaks in Android apps, in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017.

[5] A. Desnos, Androguard: Reverse engineering, malware and goodware analysis of Android applications, in *Black Hat Europe*, 2011.

[6] S. Arzt, J. Klein, and E. Bodden, DroidBench: A benchmark for Android taint analysis, in *Proceedings of the International Conference on Software Engineering Companion (ICSE-C)*, 2014.

[7] Y. Liu, Y. Wang, and J. Sun, Machine learning for software vulnerability detection: A systematic review, *IEEE Transactions on Reliability*, vol. 70, no. 3, pp. 1187–1214, 2021.

[8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[9] H. Pearce, B. Ahmad, B. Tan, B. Dolby, and S. Elbaum, Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions, in *Proceedings of the IEEE Symposium on Security and Privacy*, 2022.

[10] J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, A prompt pattern catalog to enhance prompt engineering with ChatGPT, *arXiv preprint arXiv:2302.11382*, 2023.

[11] OWASP Foundation, OWASP Mobile Application Security Verification Standard (MASVS), Available online: `https://mas.owasp.org/MASVS/`, accessed 2026.

[12] OASIS, Static Analysis Results Interchange Format (SARIF) Version 2.1.0, OASIS Standard, 27 March 2020. Available online: `https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html`.

[13] OWASP Foundation, OWASP Mobile Application Security Testing Guide (MASTG), Available online: `https://mas.owasp.org/MASTG/`, accessed 2026.

[14] V. P. Ranganath and S. Mitra, Ghera: A Repository of Android App Vulnerability Benchmarks, in *Proceedings of the 14th International Conference on Mining Software Repositories (MSR)*, 2017.

[15] X. Zhao, et al., Identifying the primary dimensions of DevSecOps: A multivocal literature review and thematic analysis, *Journal of Systems and Software*, vol. 218, 2024.

[16] L. Prates, et al., DevSecOps practices and tools, *International Journal of Information Security*, 2025.

[17] Y. Mothanna, et al., Adopting security practices in software development process, *Computers & Security*, vol. 147, 2024.